



COS711 Assignment 1

Neural Network Fundamentals: Learning Rate Warmup and Pruning Robustness

Due date: 9 October 2026

1 General instructions

This assignment is **individual** and **ungraded**. It will not be explicitly marked; instead, it serves as preparation for **Semester Test 2**, which will contain questions directly related to the concepts, experiments, and design choices you explore here. Students who engage seriously with this assignment will be substantially better prepared for Semester Test 2 than those who do not. No excuses will be accepted for incomplete engagement, and no extensions will be granted, as per the course study guide.

While this assignment will not be graded, assignment submission is compulsory as proof of work. **Students who fail to submit Assignment 1 will not be admitted to Semester Test 2.**

You must submit an archive (zip file) containing (1) a [Jupyter](#) notebook with all of your code, and (2) a short written report, as described in Section 4.

2 Neural Network Fundamentals

For this assignment, you must build and systematically investigate a multilayer perceptron (MLP) applied to a classification or regression task of your choosing (see Section 2.1). The assignment has two parts: a core investigation into MLP fundamentals (Section 2.2), and a focused research investigation into the relationship between learning rate warmup and a model's robustness to compression (Section 2.3).

2.1 Data set

Choose **one** dataset from the approved list shown in Table 1, or propose your own subject to lecturer approval. All approved datasets are hosted on the [UCI Machine Learning Repository](#) and are fetchable directly in Python via the `ucimlrepo` package (`pip install ucimlrepo`).

Whichever dataset you choose, **data preparation is a compulsory part of this assignment**. Your report must address the following for your chosen dataset:

1. **Feature relevance:** some features may not be informative. Identify and justify removal of any uninformative features.
2. **Outliers:** the data set may contain outliers. Develop and justify a sensible strategy to identify and handle them.
3. **Scaling:** apply appropriate normalisation or standardisation, and justify your choice given the feature types present.

4. **Splitting:** define your train/validation/test split strategy and justify it.
5. **Imbalance:** the classes (or target distribution, for regression) may not be balanced. Explain how this is handled during training, testing, and evaluation.

Table 1: Approved Datasets

Dataset	Domain	Task	Description
HTRU2 (Pulsar Candidates)	Radio astronomy	Binary classification (17,898 rows, 8 features)	Distinguish real pulsars from radio-frequency interference and noise, using statistics of pulse profiles and DM-SNR curves. Heavily imbalanced: only 9.16% of candidates are real pulsars.
Superconductivity	Materials science / condensed matter physics	Regression (21,263 rows, 81 features)	Predict a superconductor's critical temperature from statistical descriptors of its chemical composition (atomic mass, ionisation energy, thermal conductivity, etc.). The target distribution is strongly skewed.
Parkinson's Tele-monitoring	Neurology / biomedical signal processing	Regression (5,875 rows, 16 features)	Predict clinical UPDRS symptom scores from biomedical voice measurements (jitter, shimmer, noise-to-harmonics ratio) collected remotely from 42 patients. The smallest dataset on this list, suited to fast iteration.
Protein Tertiary Structure	Structural biology / biochemistry	Regression (45,730 rows, 9 features)	From the CASP protein-folding competitions: predict structural deviation (RMSD) of a candidate protein fold from physicochemical descriptors. The largest dataset on this list, well suited to the Part 2 warmup/pruning experiments.
Anuran Calls (MFCCs)	Bioacoustics / ecology	Multi-label classification: family / genus / species (7,195 rows, 22 features)	Frog species identification from acoustic call features (Mel-frequency cepstral coefficients (MFCCs)) recorded in the wild in Brazil and Argentina. Class imbalance across species; family/genus/species makes for a challenging hierarchical classification task.

2.1.1 Obtaining the Data

All five datasets are hosted on the UCI Machine Learning Repository and can be installed via:

```
pip install ucimlrepo
```

Use the corresponding snippet below for the dataset of your choice. Each call returns an object with `.data.features` and `.data.targets` pandas DataFrames. If you prefer a manually downloaded CSV instead, each URL below also provides a direct download link on the dataset's page.

- **HTRU2:** <https://archive.ics.uci.edu/dataset/372/htru2>

```
from ucimlrepo import fetch_ucirepo
htru2 = fetch_ucirepo(id=372)
X, y = htru2.data.features, htru2.data.targets
```
- **Superconductivity:** <https://archive.ics.uci.edu/dataset/464/superconductivity+data>

```
from ucimlrepo import fetch_ucirepo
superconductivity = fetch_ucirepo(id=464)
X, y = superconductivity.data.features, superconductivity.data.targets
```

- **Parkinson’s Telemonitoring:** <https://archive.ics.uci.edu/dataset/189/parkinsons+telemonitoring>

```
from ucimlrepo import fetch_ucirepo
parkinsons = fetch_ucirepo(id=189)
X, y = parkinsons.data.features, parkinsons.data.targets
```

- **Protein Tertiary Structure:** <https://archive.ics.uci.edu/dataset/265/physicochemical+properties+of+protein+tertiary+structure>

```
from ucimlrepo import fetch_ucirepo
protein_structure = fetch_ucirepo(id=265)
X, y = protein_structure.data.features, protein_structure.data.targets
```

- **Anuran Calls (MFCCs):** <https://archive.ics.uci.edu/dataset/406/anuran+calls+mfccs>

```
from ucimlrepo import fetch_ucirepo
anuran_calls = fetch_ucirepo(id=406)
X, y = anuran_calls.data.features, anuran_calls.data.targets
```

2.2 Core Investigation

Train an MLP baseline, then design and run a series of controlled experiments exploring **at least two** of the following axes. You must use *k-fold cross-validation* (applied to the training set) to evaluate the effect of the hyperparameters below.

- **Initialisation:** random vs. Xavier/He initialisation.
- **Activation functions:** sigmoid vs. tanh vs. ReLU (and variants).
- **Architecture:** depth vs. width trade-offs; number of hidden units.
- **Optimizers:** SGD vs. momentum vs. Adam/RMSprop; convergence speed and stability.
- **Regularisation:** weight decay, dropout, early stopping; overfitting behaviour.

For each axis you investigate, state a clear hypothesis before running your experiments, and report whether your results supported or contradicted it. Remember that simply pasting a table with numbers in it, or a graph with no explanation, yields little insight: interpretation is the more important part of the research process.

2.3 Research Investigation: Warmup and Compression Robustness

2.3.1 Background

It is a documented finding in the pruning literature that the learning rate schedule can substantially affect a network’s robustness to pruning. In their Lottery Ticket Hypothesis work, Frankle and Carbin [1] found that a learning rate warmup schedule and a constant learning rate could produce comparable accuracy for the *unpruned* network, but that the warmup schedule resulted in substantially higher accuracy as the network was *iteratively pruned*. Renda, Frankle, and Carbin [2] extended this observation in their study of learning rate rewinding. They showed that retraining a pruned network using the original learning rate schedule, rather than fine-tuning with a small constant learning rate, generally provides substantially better performance under iterative pruning.

The proposed mechanism connects two empirical observations: (1) learning rate warmup allows a network to tolerate a larger target learning rate than it otherwise could [3], and (2) learning rate has been shown to influence a network’s ability to recover from pruning, with larger learning rates sometimes yielding better recovery than smaller rates [2]. Together, these observations motivate the hypothesis that **warmup may indirectly improve pruning robustness** by enabling training at a larger effective learning rate. This causal interpretation, however, remains a *hypothesis* to be tested rather than an established result.

2.3.2 Research Task

Design a controlled experiment to test the hypothesis described above. Repeat each experiment using **at least three random seeds**, and report mean and standard deviation.

1. Train two versions of your network: one with learning rate warmup and one without warmup (using either a constant learning rate or a standard decay schedule). Choose the training configurations so that the resulting models have comparable accuracy on the *unpruned* network.
2. Apply magnitude-based pruning to both networks at one or more sparsity levels (e.g., 30%, 50%, and 70%). Use the same pruning procedure and sparsity levels for both networks.
3. Fine-tune the pruned networks using the same fine-tuning procedure and compare the recovered accuracy of the warmup and no-warmup models.
4. Report and interpret your results: did you observe the effect described above? Explain whether your results support or contradict the proposed learning rate tolerance and pruning robustness mechanism, and why.

2.3.3 Optional Extensions

For students who wish to explore further (not required, but encouraged):

- **Timing:** Does it matter *when* pruning is applied relative to the warmup phase (e.g. during warmup vs. well after it)?
- **Warmup duration:** Does a longer warmup phase yield greater pruning robustness than a short one, or is the effect binary (warmup vs. none)?
- **Distillation:** Does a teacher network trained with warmup produce a better-performing distilled student than an equivalent teacher trained without warmup?
- **Neural Architecture Search:** If you run a lightweight NAS (e.g. random search or an evolutionary search over a small search space), does the training schedule used to *evaluate* candidate architectures during search change which architectures are selected, even when all candidates are later retrained fully with a different schedule?

2.3.4 Suggested Reading

See references [1, 2, 3] at the end of this document. In particular, Appendix I.5 of [1] discusses warmup iterations directly.

3 Notes

- Implementation
 - This is your opportunity to learn how to build an MLP from scratch.
 - Try implementing some or all of the concepts from first principles.
 - You are required to use Python programming language for this assignment, and submit your code in Jupyter notebook format.
 - You may use a machine learning/neural network library/framework.
- Written Report
 - You must report on all data preparation steps taken.
 - You must state a hypothesis for each experiment before reporting its results.
 - You must clearly report and interpret your warmup/pruning comparison, including plots of accuracy vs. sparsity for both conditions.

4 Deliverables

1. A short technical report (2–4 pages, excluding code) covering the core investigation and the research investigation, including learning curves, comparison tables/plots, and a written interpretation of each experiment.
2. A Jupyter notebook containing your code, sufficient to reproduce your reported results.

Package both files into a single *zip* archive and upload it to the appropriate assignment slot on ClickUP. Multiple uploads are allowed; only the last one will be considered. This assignment must be completed **individually**. The deadline is **9 October 2026**.

5 A Note on Openness

This assignment is deliberately open-ended. You are not expected to explore every axis listed, nor to complete every optional extension. You are expected to demonstrate that you can design a fair experiment, state a hypothesis, interpret evidence honestly (including when it contradicts your hypothesis), and connect your empirical findings back to the underlying theory covered in lectures. Depth of understanding on fewer experiments is preferred over shallow coverage of many.

References

- [1] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, New Orleans, LA, USA, 2019.
- [2] A. Renda, J. Frankle, and M. Carbin, “Comparing rewinding and fine-tuning in neural network pruning,” in *Proc. Int. Conf. Learning Representations (ICLR)*, Addis Ababa, Ethiopia, 2020.
- [3] D.S. Kalra and M. Barkeshli, “Why Warmup the Learning Rate? Underlying Mechanisms and Improvements,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Vancouver, Canada, 2024.